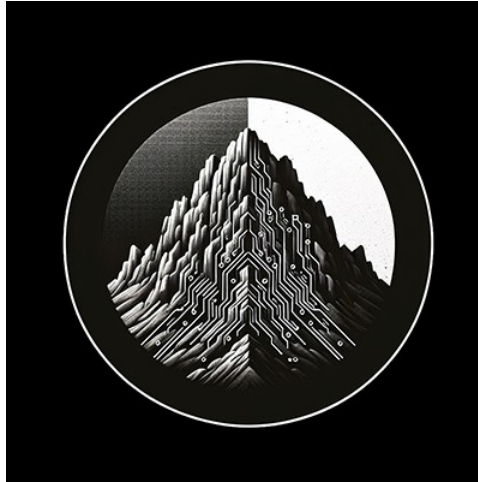




RTL Design Sherpa

STREAM Characterization Monbus Compression Report

June 2026

Table of Contents

1 Readme.....	4
2 compression/ — monbus half-beat compression characterization.....	4
2.1 What “compression” means here.....	4
2.2 Headline result (real captured traffic).....	5
2.3 Live on-board confirmation (all-fixes bitstream).....	5
2.4 Half-beat wire format (the 32-bit slot).....	6
2.5 Bitstream under test.....	6
2.6 Reproduce.....	7
2.7 Files.....	8
2.8 See also.....	8

List of Tables

No tables in this document.

1 Readme

Generated: 2026-06-15

2 compression/ — monbus half-beat compression characterization

Companion to reports/perf/. Where perf measures DMA bus utilization, this measures how well the **monbus compressor** + **half-beat packer** shrink the monitor-event stream on real captured traffic.

- Format spec / golden codec:
`bin/TBClasses/monbus/monbus_compressor.py`
- Source capture dataset:
`reports/compression_dataset/desc_axi_16desc_8ch_1MB.json`
- Offline measurement tool:
`flows-stream-bridge/host/monbus_halfbeat_model.py`
- On-board live measurement: `flows-stream-bridge/host/hb_measure.py`

2.1 What “compression” means here

Each monitor record is a 128-bit packet + 64-bit timestamp. The raw (Tier-0) escape costs $3 \times 64\text{-bit beats}$ per record. The compressor removes that cost two ways:

1. **Tier-1 templating** (64-bit codec): a CAM-hit record collapses to **1 beat**.
Structural ceiling = $1 - 1/3 = 66.7\%$, reached only when every record is a hit.
2. **Half-beat packing** (32-bit codec): two compressible records share one beat (**0.5 beat/record**), raising the ceiling to $1 - 0.5/3 = 83.3\%$.

The realized reduction is dominated by the Tier-1 hit rate h , not the slot size:

$$\text{reduction} = h \cdot (1 - b_t / 3)$$

current ($b_t = 1.0$):	$h \cdot 0.667$	ceiling 66.7%
half-beat ($b_t = 0.5$):	$h \cdot 0.833$	ceiling 83.3%

So 80% reduction needs $h \approx 0.96$. The point of measuring on real traffic is to find the actual h and the fraction of hits that fit a 32-bit half-slot (vs. needing a full 64-bit beat).

2.2 Headline result (real captured traffic)

Replaying the committed encoder CAM logic over the **682-record on-board capture** (desc_axi_16desc_8ch_1MB.json, the descriptor-fetch read-bus monitor across 8 channels × 16 descriptors × 1 MB):

Codec	Beats	Reduction	Notes
raw baseline	2046	—	682 records × 3 beats
current 64-bit	758	63.0 %	1 beat/hit, 3/escape
half-beat 32-bit	498	75.7 %	two hits share a beat

Stream composition: $h = 0.944$ Tier-1 hit rate — **521 half-fit, 123 full-only, 38 raw escapes**. Half-beat packing converts the 63.0 % codec into 75.7 % on this trace: a **12.7-percentage-point** gain, or **260 fewer beats** drained over UART for the same monitor coverage.

Why 75.7 % and not the 83.3 % ceiling: $h = 0.944$ caps half-beat at $0.944 \times 0.833 = 78.6 \%$, and of the hits, 123/644 fit only a full 64-bit beat (event_data or delta_ts too wide for a 23-bit half-payload), plus lone-half round-up. The gap to ceiling is real-traffic structure, not codec overhead.

2.3 Live on-board confirmation (all-fixes bitstream)

The numbers above are the offline golden codec. The same codec is **synthesized into the bitstream** (USE_MON_COMPRESSION=USE_MON_HALFBEAT=1). hb_measure.py runs the compressed workload, drains the real on-chip trace SRAM, and decodes the FPGA-emitted 64-bit slots with the golden Decoder — every run below round-trips clean (decode=OK), proving the HW slot stream is bit-exact decodable. Monitor preset debug-compl, reset between runs:

config	records	half-beat beats	raw-equiv	reduction
4desc_1ch_1MB	16	23	48	52.1 %
8desc_1ch_1MB	32	35	96	63.5 %
4desc_4ch_1MB	64	94	192	51.0 %
8desc_4ch_1MB	125	140	375	62.7 %
8desc_8ch_1MB	212	261	636	59.0 %

config	records	half-beat beats	raw-equiv	reduction
1MB				

These live runs are **warm-up-dominated and trace-depth-limited**: the bulk-trace SRAM holds only 2048 beats, so the largest clean workload is ~200 records — small enough that the one-time CAM-miss escapes (one raw 3-beat escape per *first* occurrence of each (channel, event) template) are a large fraction of the stream. Adding channels makes this worse, not better: each channel contributes its own cold templates, so 8desc_8ch (8 cold templates × 8 channels) lands at 59.0 % while 8desc_1ch reaches 63.5 %. As the record count grows the escapes amortize and the realized reduction climbs toward the full-capture **75.7 %** (682 records). The live data confirms the HW codec is correct and functional; the offline model on the full capture is the representative reduction figure.

2.4 Half-beat wire format (the 32-bit slot)

A half-pair beat is {tag[63:60]=0x4, slotA[59:30], slotB[29:0]} — two 30-bit slots. Each slot = sub[1:0] + idx[4:0] + 23-bit payload:

```
HALF-A (absolute):  sub=1 | idx[4:0] | delta_ts[9:0]      |
event_data[12:0]
HALF-C (delta):    sub=2 | idx[4:0] | delta_ts[9:0]      |
ed_delta[11:0] signed
NOP (lone-half pad):sub=0
```

- delta_ts < 1024 cycles (~10.2 μs @ 100 MHz) for either half form.
- HALF-A carries event_data < 2¹³; HALF-C carries a signed ed_delta | · | < 2¹² for counters / sequential addresses.
- A record that is Tier-1 but fits neither half form stays a full 64-bit beat. A lone trailing half rounds up to a full beat (flushed before any full/raw beat and at end-of-stream).

These widths are locked in monbus_halfbeat_model.py (lines 50–62) and mirrored bit-exact by rtl/amba/shared/monbus_halfbeat_packer.sv.

2.5 Bitstream under test

All numbers are for the all-timing-fixes bitstream (see reports/perf/README.md — WNS +0.048 ns @ 100 MHz, xc7a100t), built with the compressor + packer enabled:

```
USE_MON_COMPRESSION = 1      # monbus_compressor (Tier-0/1)
USE_MON_HALFBEAT    = 1      # monbus_halfbeat_packer (32-bit slots)
USE_MON_CAM_PIPELINE= 1      # pipelined match CAM (timing)
```

The on-chip half-beat packer is **bit-exact** to the Python golden: `val/amba/test_monbus_halfbeat.py` and `test_monbus_compressor.py` verify the RTL emits the identical slot stream the model produces for the same record sequence. Therefore the offline reduction above equals what the FPGA emits for that record stream — the model is not an estimate, it is the codec.

2.6 Reproduce

`source env_python`

```
# 1. Offline, bit-exact, on the locked real capture (no board needed):
cd projects/NexysA7/stream_characterization/flows-stream-bridge
python3 - <<'PY'
import json, sys; sys.path.insert(0, 'host')
from monbus_halfbeat_model import classify, _print
doc =
json.load(open('../reports/compression_dataset/desc_axi_16desc_8ch_1MB
.json'))
recs = [(int(r['packet'], 16), int(r['timestamp'], 16)) for r in
doc['records']]
_print(f'real capture ({len(recs)} records)', classify(recs))
PY

# 2. Synthetic envelope (event_data shape sweep, ceilings 66.7 /
83.3%):
python3 host/monbus_halfbeat_model.py

# 3. Live on-board: run the compressed workload, drain the trace SRAM,
#   decode with the golden, report reduction. Keep the workload small
—
#   the bulk-trace SRAM is 2048 beats; a 1 MB debug-all run overflows
it.
python3 host/hb_measure.py --port /dev/ttyUSB2 \
--channels 1 --config 1desc_1ch_1MB --mon-config debug-compl
```

Trace-SRAM bound. `hb_measure.py` reads the 2048-beat bulk-trace SRAM and refuses to report on an overflowed trace (the write pointer wraps, so a partial count would be wrong). For a clean live count use a lighter `--mon-config` (e.g. `perf-mon / debug-compl`) and/or a small workload so the compressed beat count stays under 2048. For full-workload reduction, use the offline model on a `per_source_capture` dump (method 1) — it has no depth limit and is bit-exact to the RTL.

2.7 Files

File	Source	Notes
reports/ compression_dataset/ desc_axi_16desc_8ch_1 MB.json	on-board capture, 682 records	locked dataset; see its README_COMPRESSION_DA TASET.md
reports/ compression_dataset/ README_COMPRESSION_DA TASET.md	—	full Tier-0/1 format handoff (CAM=32, idx=5b)

2.8 See also

- reports/perf/README.md — DMA bus-utilization characterization.
- bin/TBClasses/monbus/monbus_compressor.py — locked codec (Tier-0/1 + half-beat).
- rtl/amba/shared/monbus_halfbeat_packer.sv — RTL half-beat packer.
- rtl/amba/shared/monbus_compressor.sv — RTL compressor (CAM_PIPELINE, HALF_BEAT_EN).